Studio Nx

JS 09 — setFieldState()

2026-10-18

JS 09 — `setFieldState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101, FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	08 `logout()` (P1-02의 끝) · 10 `initPasswordToggles()`

1. 하는 일

입력칸 하나의 **상태**(초록, 빨강, 기본)와 그 아래의 **문구**를 바꿉니다.

호출	화면 (join.html의 이메일 칸)
`setFieldState(field, 'ok', '✓ Available')`	밑줄이 **초록**이 되고, 아래에 초록 글씨로 "✓ Available"이 나옵니다
`setFieldState(field, 'error', 'This email is already in use.')`	밑줄이 **빨강**이 되고, 아래에 빨강 글씨가 나옵니다. 화면 읽기 프로그램은 "잘못된 값"으로 읽습니다
`setFieldState(field, null)`	밑줄이 기본 색(보라)으로 돌아가고, 문구는 처음 HTML에 있던 안내 글로 돌아갑니다 (join.html은 빈 문자열)

`field`는 입력칸을 감싼 `.field` 요소입니다 (예: `#join-email-field`).

```
<div class="field" id="join-email-field">          ← is-ok / is-error 가 여기 붙음
  <input class="field__control" id="join-email" …>  ← aria-invalid="true" 가 여기 붙음
  <p class="field__msg" id="join-email-msg"></p>     ← 문구가 여기 들어감
</div>
```

2. 언제 쓰나

P1-07의 다른 함수들이 칸마다 결과를 보여 줄 때 이 함수를 부릅니다. 그래서 가장 먼저 만들었습니다.

부르는 곳 (P1-07)	칸	예
`initDuplicateChecks()`	이메일, 닉네임	Check를 누르면 ✓ Available 또는 This email is already in use.
`initPasswordRules()`	비밀번호, 비밀번호 확인	Use only English letters, numbers and symbols. / Passwords don't match.
`initJoinForm()`	서버가 422로 돌려준 칸	`details.nickname`이 있으면 닉네임 칸이 빨강이 됩니다

색은 `forms.css`가 정합니다. 이 함수는 **클래스와 속성만** 바꿉니다.

붙이는 것	`forms.css`가 하는 일
`.field.is-ok`	밑줄과 문구를 초록(`--c-ok`)으로 만듭니다
`.field.is-error` + `aria-invalid="true"`	밑줄과 문구를 빨강(`--c-error`)으로 만듭니다

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js`에는 아직 import가 없습니다 (다음 함수들에서 `api.js`와 `auth.js`를 가져올 예정입니다). **이 함수는 import가 필요하지 않습니다.** `querySelector`, `classList`, `dataset`은 브라우저의 기본 기능입니다.

함수

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.remove('is-ok', 'is-error');
  if (control) control.removeAttribute('aria-invalid');

  if (state === 'ok') {
    field.classList.add('is-ok');
  } else if (state === 'error') {
    field.classList.add('is-error');
    if (control) control.setAttribute('aria-invalid', 'true');
  }

  if (msg) {
    if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent; // 처음 안내 글을 기억
    msg.textContent = message === undefined ? msg.dataset.hint : message;
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위의 전통 문법입니다)

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.toggle('is-ok', state === 'ok');
  field.classList.toggle('is-error', state === 'error');
  if (state === 'error') control?.setAttribute('aria-invalid', 'true');
  else control?.removeAttribute('aria-invalid');

  if (msg) {
    msg.dataset.hint ??= msg.textContent;
    msg.textContent = message ?? msg.dataset.hint;
  }
}
```

최신 문법	실제 코드 (전통 문법)	뜻
`control?.setAttribute(…)`	`if (control) control.setAttribute(…)`	`control`이 있을 때만 실행합니다

최신 문법	실제 코드 (전통 문법)	뜻
`msg.dataset.hint ??= msg.textContent`	`if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent`	비어 있을 때만 넣습니다
`message ?? msg.dataset.hint`	`message === undefined ? msg.dataset.hint : message`	값이 없으면 대신 쓸 값입니다 (`??`는 `null`도 "없음"으로 봅니다)
`classList.toggle('is-ok', 조건)`	`remove`한 뒤 조건에 맞으면 `add`	조건이 참이면 붙이고, 거짓이면 뗍니다

문법 풀이

import 문법	뜻
`export function`	다른 파일(`join.js` 등)이 `import { setFieldState } from '../core/auth-ui.js'`로 가져다 씁니다

식	뜻
`if (!field) return;`	`field`가 없으면(`null` 등) 아무것도 하지 않고 끝냅니다
`field.querySelector('.field__control')`	`field` **안에서** 이 클래스를 가진 첫 요소를 찾습니다. 없으면 `null`입니다
`classList.remove('is-ok', 'is-error')`	두 클래스를 한 번에 뗍니다. 없어도 오류가 나지 않습니다
`setAttribute('aria-invalid', 'true')`, `removeAttribute(…)`	HTML 속성을 붙이거나 뗍니다
`msg.dataset.hint`	HTML의 `data-hint="…"` 속성입니다. 이 함수를 처음 부를 때 만들어서 안내 글을 보관합니다
`조건 ? A : B`	조건이 참이면 A, 아니면 B입니다 (오래된 전통 문법인 삼항 연산자)
`message === undefined`	세 번째 인자를 **넘기지 않았는지** 확인합니다. `''`(빈 문자열)를 넘기면 빈 문자열로 바꿉니다

4. 흐름

1. `field`가 없으면 끝냅니다.
2. 상태 표시(`is-ok`, `is-error`, `aria-invalid`)를 모두 지웁니다.
3. `'ok'`이면 초록, `'error'`이면 빨강과 `aria-invalid`를 붙입니다.
4. 문구를 바꿉니다. 넘긴 문구가 있으면 그것을, 없으면 처음의 안내 글을 씁니다.

5. 왜 이렇게

선택	이유
먼저 모두 지운 뒤 붙임	`ok`에서 `error`로 바뀔 때 `is-ok`가 남아서 초록과 빨강이 섞이는 일이 없습니다
색은 CSS에 두고, JS는 클래스만 바꿈	디자인을 바꿀 때 JS를 건드리지 않아도 됩니다. 같은 상태 규칙을 모든 폼(`forms.css`)이 씁니다
`aria-invalid="true"`	색을 볼 수 없는 사람도 화면 읽기 프로그램으로 "잘못된 입력"임을 알 수 있습니다. 문구 칸은 `aria-live="polite"`라서 바뀌면 읽어 줍니다
처음 안내 글을 `data-hint`에 보관	"2–20 characters" 같은 안내가 있는 칸에서 오류를 지우면 안내 글이 돌아옵니다. HTML을 고치지 않아도 됩니다

선택	이유
`field`, `control`, `msg`가 없어도 오류가 나지 않음	칸 모양이 조금 다른 폼(로그인 모달 등)에서도 안전하게 쓸 수 있습니다

6. 시험 (실제 join.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 11개)

P1-02의 Node 시험과 달리, 이 함수는 실제 HTML과 CSS가 있어야 하므로 **브라우저 안에서** 시험했습니다 (Sentry를 끈 8001번 개발 서버).

시험	확인한 것
처음 상태	클래스가 없고, `aria-invalid`도 없으며, 문구는 빈 문자열입니다
ok, error, null, error→ok (4개)	클래스, `aria-invalid`, 문구가 표대로 바뀌고, 바뀔 때 이전 상태가 남지 않습니다
실제 밑줄 색	error는 `rgb(255, 138, 138)`, ok는 `rgb(127, 224, 168)`, 기본은 `rgba(163, 131, 255, 0.45)`입니다. `forms.css`가 실제로 적용되었습니다
안내 글이 있는 칸 (4개)	error 뒤에 null이면 "2–20 characters"로 복원되고, 문구 없이 ok여도 안내 글이 나오며, 빈 문자열을 넘기면 빈 문자열이 됩니다
`field`가 `null`	오류가 나지 않습니다

시험 중에 한 번 실패한 것: 처음에는 세 상태의 밑줄 색이 모두 같게 나왔습니다. `forms.css`의 색 전환 효과(`transition`) 때문에, 바꾼 직후에는 아직 이전 색이었습니다. 함수의 문제가 아니라 시험이 너무 빨리 읽은 것이었으므로, 0.6초 기다린 뒤 읽도록 고쳤습니다.

11개 모두 통과했습니다.